# The SQL Join Clause

The SQL **Join** clause is used to combine data from two or more tables in a database. When the related data is stored across multiple tables, joins help you to retrieve records combining the fields from these tables using their foreign keys.

The part of the Join clause that specifies the columns on which records from two or more tables are joined is known as **join-predicate**. This predicate is usually specified along with the ON clause and uses various comparison operators such as, <, >, <>, <=, >=, !=, BETWEEN, LIKE, and NOT etc. We can also connect multiple join predicates with logical operators AND, OR, and NOT.

*We can use JOINs along with update and delete, SQL queries to update and delete records from across multiple tables. When you retrieve a table using joins, the resultant table displayed is not stored anywhere in the database.*

Syntax

Following is the basic syntax of a the SQL JOIN CLAUSE −

SELECT column_name(s)

FROM table1

JOIN table2;

Example

Assume we have created a CUSTOMERS table that contains details of the customers of an organization using the following query −

Open Compiler

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2),
   PRIMARY KEY (ID)
```

);

Now insert values into this table using the INSERT statement as follows −

Open Compiler

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),

(2, 'Khilan', 25, 'Delhi', 1500.00 ),

(3, 'Kaushik', 23, 'Kota', 2000.00 ),

(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),

(5, 'Hardik', 27, 'Bhopal', 8500.00 ),

(6, 'Komal', 22, 'Hyderabad', 4500.00 ),

(7, 'Muffy', 24, 'Indore', 10000.00 );

The CUSTOMERS table will be created as follows −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Following is another table ORDERS which contains the order details made by the customers.

Open Compiler

```
CREATE TABLE ORDERS (
   OID INT NOT NULL,
   DATE VARCHAR (20) NOT NULL,
   CUSTOMER_ID INT NOT NULL,
   AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The ORDERS table will be created as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Following query performs the join operation on the tables CUSTMERS and ORDERS −

Open Compiler

```
SELECT ID, NAME, AGE, AMOUNT
FROM CUSTOMERS
JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Output

By executing the query above, the resultant table is displayed and contains the values present in ID, NAME, AGE fields of CUSTOMERS table and AMOUNT field of ORDERS table.

| ID | NAME | AGE | AMOUNT |
|---|---|---|---|
| 3 | Kaushik | 23 | 3000 |
| 3 | Kaushik | 23 | 1500 |
| 2 | Khilan | 25 | 1560 |
| 4 | Chaitali | 25 | 2060 |

Fullscreen

## Types of joins in SQL

SQL provides various types of Joins that are categorized based on the way data across multiple tables are joined together. They are listed as follows −

## Inner Join

An INNER JOIN is the default join which retrieves the intersection of two tables. It compares each row of the first table with each row of the second table. If the pairs of these rows satisfy the join-predicate, they are joined together.

## Outer Join

An Outer Join retrieves all the records in two tables even if there is no counterpart row of one table in another table, unlike Inner Join. Outer join is further divided into three subtypes - Left Join, Right Join and Full Join.

Following are the different types of outer Joins −

- LEFT JOIN − returns all rows from the left table, even if there are no matches in the right table.
- RIGHT JOIN − returns all rows from the right table, even if there are no matches in the left table.
- FULL JOIN − returns rows when there is a match in one of the tables.

Other Joins

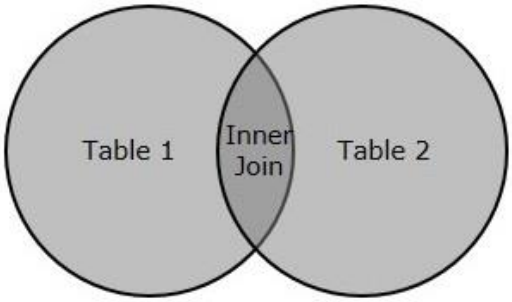
In addition to these there are two more joins −

- SELF JOIN − is used to join a table to itself as if the table were two tables, temporarily renaming at least one table in the SQL statement.
- CROSS Join − returns the Cartesian product of the sets of records from the two or more joined tables.

## The SQL Inner Join

The **SQL Inner Join** is a type of join that combines multiple tables by retrieving records that have matching values in both tables (in the common column).

It compares each row of the first table with each row of the second table, to find all pairs of rows that satisfy the join-predicate. When the join-predicate is satisfied, the column values from both tables are combined into a new table.



*The Inner Join is also referred as **Equijoin**. It is the default join; i.e., even if the "**Join**"keyword is used instead of "**Inner Join**", tables are joined using matching records of common columns.*

Explanation

Let us look at an example scenario to have a better understanding.

Suppose we have the information of employees in a company divided between two tables namely EmpDetails and Marital status. Where,

- EmpDetails table holds details like Employee ID, Name and Salary.
- MaritalStatus table holds the details Employee ID, Age, and Marital Status.

**EmpDetails**

| ID | Name | Salary |
|---|---|---|
| 1 | John | 40000 |
| 2 | Alex | 25000 |
| 3 | Simon | 43000 |

**MaritalStatus**

| ID | Name | Status |
|---|---|---|
| 1 | John | Married |
| 3 | Simon | Married |
| 4 | Stella | Unmarried |

| ID | Name | Salary | Status |
|---|---|---|---|
| 1 | John | 40000 | Married |
| 3 | Simon | 43000 | Married |

When we perform the Inner Join operation on these two tables based on the join-predicate **EmpDetails.EmpID = MaritalStatus.EmpID**, the resultant records hold the following info: ID, Name, Salary, Age and, Status of the matched records.

Syntax

Following is the basic syntax of SQL Inner Join −

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

Example

Assume we have created a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary etc., using the following query −

Open Compiler

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

Now insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |

| | | | | |
|---|---|---|---|---|
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Let us create another table ORDERS, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
   OID INT NOT NULL,
   DATE VARCHAR (20) NOT NULL,
   CUSTOMER_ID INT NOT NULL,
   AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Let us now combine these two tables using the **Inner Join query** as shown below −

Open Compiler

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
INNER JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Output

The result of this query is obtained as follows −

| ID | NAME | AMOUNT | DATE |
|---|---|---|---|
| 3 | Kaushik | 3000.00 | 2009-10-08 00:00:00 |
| 3 | Kaushik | 1500.00 | 2009-10-08 00:00:00 |
| 2 | Khilan | 1560.00 | 2009-11-20 00:00:00 |
| 4 | Chaitali | 2060.00 | 2008-05-20 00:00:00 |

## Joining Multiple Tables Using Inner Join

Until now, we have only learnt how to join two tables using Inner Join. However, we can also join as many tables as possible, using Inner Join, by specifying the condition (with which these tables are to be joined).

Syntax

Following is the syntax to join more than two tables using Inner Join −

```
SELECT column1, column2, column3...
FROM table1  INNER JOIN  table2
ON condition_1
INNER JOIN table3
ON condition_2
....
....
INNER JOIN tableN
ON condition_N;
```

Note that, even in this case, only two tables can be joined together on a single condition. This process is done sequentially until all the tables are combined.

Example

Let us make use of the previous tables CUSTOMERS and ORDERS along with a new table **EMPLOYEE**. We will create the EMPLOYEE table using the query below −

Open Compiler

```
CREATE TABLE EMPLOYEE (
   EID INT NOT NULL,
   EMPLOYEE_NAME VARCHAR (30) NOT NULL,
   SALES_MADE DECIMAL (20)
```

);

Now, we can insert values into this empty tables using the INSERT statement as follows −

Open Compiler

INSERT INTO EMPLOYEE VALUES

(102, 'SARIKA', 4500),

(100, 'ALEKHYA', 3623),

(101, 'REVATHI', 1291),

(103, 'VIVEK', 3426);

The details of EMPLOYEE table can be seen below.

| EID | EMPLOYEE_NAME | SALES_MADE |
|---|---|---|
| 102 | SARIKA | 4500 |
| 100 | ALEKHYA | 3623 |
| 101 | REVATHI | 1291 |
| 103 | VIVEK | 3426 |

Using the following query, we can combine three tables CUSTOMERS, ORDERS and EMPLOYEE.

SELECT OID, DATE, AMOUNT, EMPLOYEE_NAME FROM CUSTOMERS

INNER JOIN ORDERS

ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID

INNER JOIN EMPLOYEE

ON ORDERS.OID = EMPLOYEE.EID;

Output

The result of the inner join query above is shown as follows −

| OID | DATE | AMOUNT | EMPLOYEE_NAME |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3000.00 | SARIKA |
| 100 | 2009-10-08 00:00:00 | 1500.00 | ALEKHYA |
| 101 | 2009-11-20 00:00:00 | 1560.00 | REVATHI |
| 103 | 2008-05-20 00:00:00 | 2060.00 | VIVEK |

## Inner Join with WHERE Clause

Clauses in SQL work with the purpose of applying constraints while retrieving data using SQL queries. There are various clauses that SQL uses to constraint the data; such as WHERE clause, GROUP BY clause, ORDER BY clause, UNION clause etc.

The WHERE clause is used to filter the data from tables. This clause specifies a condition to retrieve only those records that satisfy it.

Inner Join uses WHERE clause to apply more constraints on the data to be retrieved. For instance, while retrieving the employee records of an organization, if we only want to check the data of employees that earn more than 25000 in a month, we need to specify a WHERE condition (salary > 25000) to retrieve only those employee records.

Syntax

The syntax of Inner Join when used with WHERE clause is given below –

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

Example

In this example we are joining the tables CUSTOMERS and ORDERS using the inner join query and we are applying some constraints on the result using the WHERE clause.

Here, we are retrieving the ID and NAME from the CUSTOMERS table and DATE and AMOUNT from the ORDERS table where the amount paid is higher than 2000.

Open Compiler

```
SELECT ID, NAME, DATE, AMOUNT FROM
CUSTOMERS INNER JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
WHERE ORDERS.AMOUNT > 2000.00;
```

Output

The resultant table after applying the where clause with inner join contains the rows that has AMOUNT values greater than 2000.00 −

| ID | NAME | DATE | AMOUNT |
|---|---|---|---|
| 3 | Kaushik | 2009-10-08 00:00:00 | 3000.00 |
| 4 | Chaitali | 2008-05-20 00:00:00 | 2060.00 |

*Joins are used to retrieve records from two or more tables based on a logical relation between them. This relation is defined using a join condition. As we discussed in the previous chapters, there are two types of Joins −*

- *Inner Join*
- *Outer Join*

Left Join is a type of outer join that retrieves all the records from the first table and matches them to the records in second table. First of all, let us understand what is outer join.

## What is Outer Join?

Outer Join is used to join multiple database tables into a combined result-set, that includes all the records, even if they don't satisfy the join

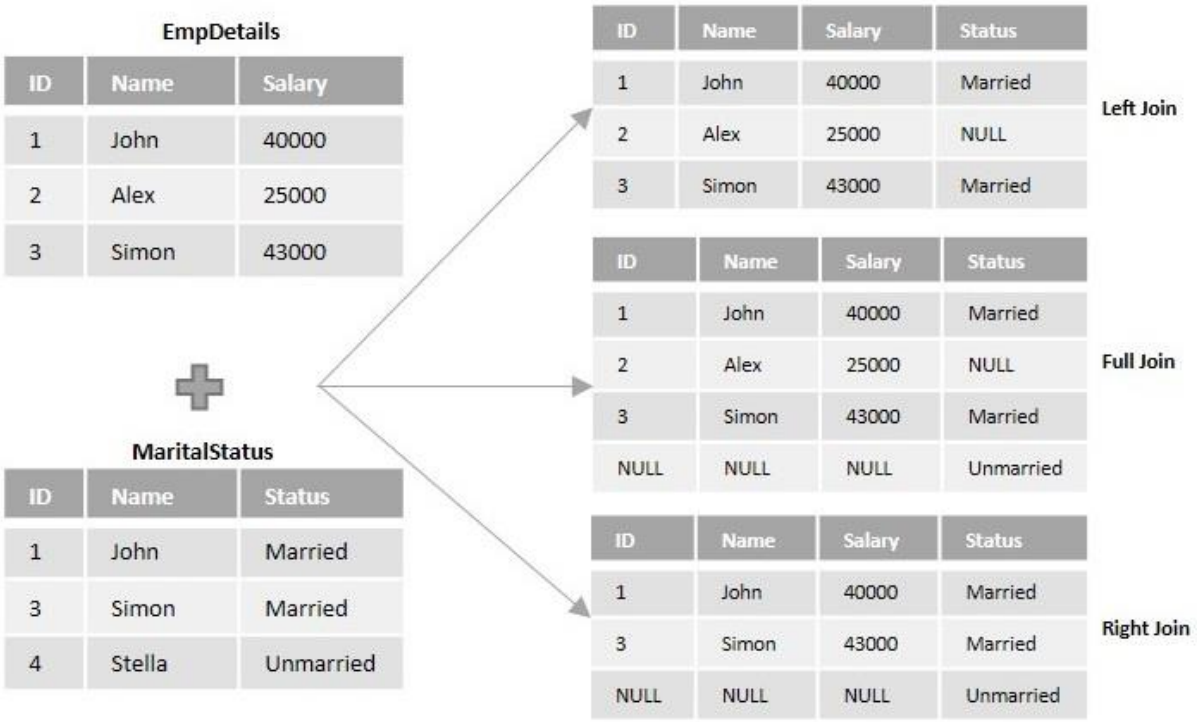
condition. NULL values are displayed against these records where the join condition is not met.

This scenario only occurs if the left table (or the first table) has more records than the right table (or the second table), or vice versa.

There are three types of outer joins, namely −

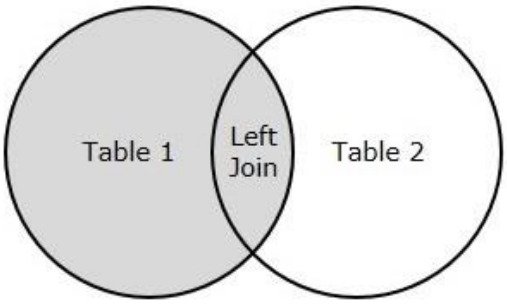
- ***Left (Outer) Join:*** *Retrieves all the records from the first table, Matching records from the second table and NULL values in the unmatched rows.*
- ***Right (Outer) Join:*** *Retrieves all the records from the second table, Matching records from the first table and NULL values in the unmatched rows.*
- ***Full (Outer) Join:*** *Retrieves records from both the tables and fills the unmatched values with NULL.*

Following diagram illustrates various outer joins between two tables namely, EmpDetails and MaritalStatus. Here, the join operation is presumed based on the join-predicate **EmpDetails.EmpID = MaritalStatus.EmpID.**



## The SQL Left Join

Left Join or Left Outer Join in SQL combines two or more tables, where the first table is returned wholly; but, only the matching record(s) are retrieved from the consequent tables. If zero (0) records are matched in the consequent tables, the join will still return a row in the result, but with NULL in each column from the right table.




*If the number of rows in first table is less than the number of rows in second table, the rows in second table that do not have any counterparts in the first table will be discarded from the result.*

Syntax

Following is the basic syntax of Left Join in SQL −

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Example

To understand this query better, let us create some tables in an existing database and join them using Left Join or Left Outer Join.

Assume we have created a table named **CUSTOMERS**, which contains the personal details of customers including their name, age, address and salary, using the following query.

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

Now insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Let us create another table **ORDERS**, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
   OID INT NOT NULL,
   DATE VARCHAR (20) NOT NULL,
   CUSTOMER_ID INT NOT NULL,
   AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Following **left join query**, retrieves the details of customers who made an order at the specified date and who did not. If there is no match found, the query below will return NULL in that record.

Open Compiler

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
LEFT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Output

The resultant table is obtained as −

| ID | NAME | AMOUNT | DATE |
|---|---|---|---|
| 1 | Ramesh | NULL | NULL |
| 2 | Khilan | 1560.00 | 2009-11-20 00:00:00 |
| 3 | Kaushik | 1500.00 | 2009-10-08 00:00:00 |
| 3 | Kaushik | 3000.00 | 2009-10-08 00:00:00 |
| 4 | Chaitali | 2060.00 | 2008-05-20 00:00:00 |
| 5 | Hardik | NULL | NULL |

| | | | |
|---|---|---|---|
| 6 | Komal | NULL | NULL |
| 7 | Muffy | NULL | NULL |

As we can see in the table above, only Khilan, Kaushik and Chaitali made purchases on the mentioned dates in ORDERS table; hence, the records are matched. The other customers in CUSTOMERS table did not make purchases on the specified dates, so the records are returned as NULL.

Joining Multiple Tables with Left Join

Similar to the Inner Join query, Left Join also joins multiple tables where the first table is returned as it is and the remaining tables are matched with the rows in the first table. If the records are not matched, NULL is returned.

The syntax to join multiple tables using Left Join is given below −

```
SELECT column1, column2, column3...
FROM table1
LEFT JOIN table2
ON condition_1
LEFT JOIN table3
ON condition_2
....
....
LEFT JOIN tableN
ON condition_N;
```

Example

To demonstrate Left Join with multiple tables, let us consider the previously created tables CUSTOMERS and ORDERS. In addition to these we will create the EMPLOYEE table using the following query −

Open Compiler

```
CREATE TABLE EMPLOYEE (
  EID INT NOT NULL,
```

```
  EMPLOYEE_NAME VARCHAR (30) NOT NULL,
  SALES_MADE DECIMAL (20)
);
```

Now, we can insert values into this empty tables using the INSERT statement as follows −

Open Compiler

```
INSERT INTO EMPLOYEE VALUES
(102, 'SARIKA', 4500),
(100, 'ALEKHYA', 3623),
(101, 'REVATHI', 1291),
(103, 'VIVEK', 3426);
```

The **EMPLOYEE** table consists of the details of employees in an organization and sales made by them.

| EID | EMPLOYEE_NAME | SALES_MADE |
|---|---|---|
| 102 | SARIKA | 4500 |
| 100 | ALEKHYA | 3623 |
| 101 | REVATHI | 1291 |
| 103 | VIVEK | 3426 |

Following query joins the CUSTOMERS, ORDERS and EMPLOYEE tables using the left join –

```
SELECT CUSTOMERS.ID, CUSTOMERS.NAME,
ORDERS.DATE, EMPLOYEE.EMPLOYEE_NAME
FROM CUSTOMERS
LEFT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
LEFT JOIN EMPLOYEE
ON ORDERS.OID = EMPLOYEE.EID;
```

Through this query, we will display the id, name of the customer along with the date on which the orders are made and the name of the employee who sold the item.

Output

The resultant table is obtained as follows −

| ID | NAME | DATE | EMPLOYEE_NAME |
|---|---|---|---|
| 1 | Ramesh | NULL | NULL |
| 2 | Khilan | 2009-11-20 00:00:00 | REVATHI |
| 3 | Kaushik | 2009-10-08 00:00:00 | ALEKHYA |
| 3 | Kaushik | 2009-10-08 00:00:00 | SARIKA |
| 4 | Chaitali | 2008-05-20 00:00:00 | VIVEK |
| 5 | Hardik | NULL | NULL |
| 6 | Komal | NULL | NULL |
| 7 | Muffy | NULL | NULL |

As we can see in the table above, the customer Kaushik made three orders, in which two are sold by employee Alekhya and one is sold by

Sarika. Khilan and Chaitali made one order each, that are sold by Revathi and Vivek respectively. The dates on which these orders are made will also be displayed. If the orders are not made on the specific dates, NULL is returned.

## Left Join with WHERE Clause

Along with the ON clause, a WHERE clause can also be applied on the obtained result-set after Left Join is implemented. Doing this will filter the data further.

Syntax

The syntax of Left Join when used with WHERE clause is given below −

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

Example

Records in the combined database tables can be filtered using the WHERE clause. Consider the previous two tables CUSTOMERS and ORDERS; and join them using the left join query by applying some constraints using the WHERE clause.

Open Compiler

```
SELECT ID, NAME, DATE, AMOUNT FROM CUSTOMERS
LEFT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
WHERE ORDERS.AMOUNT > 2000.00;
```

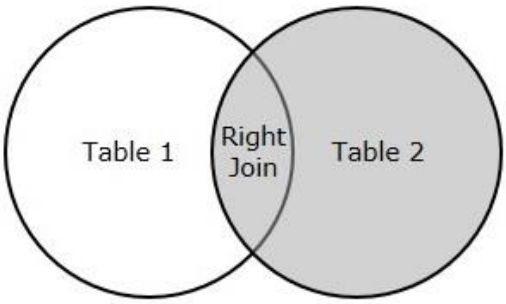
The resultant table after applying the where clause with left join contains the rows that has amount values greater than 2000.00 −

| ID | NAME | DATE | AMOUNT |
|---|---|---|---|
| 3 | Kaushik | 2009-10-08 00:00:00 | 3000.00 |

| | | | |
|---|---|---|---|
| 4 | Chaitali | 2008-05-20 00:00:00 | 2060.00 |

## The SQL Right Join

The **Right Join** or **Right Outer Join** query in SQL returns all rows from the right table, even if there are no matches in the left table. In short, a right join returns all the values from the right table, plus matched values from the left table or NULL in case of no matching join predicate.




*If the ON clause matches zero records in the left table; the join will still return a row in the result, but with a NULL value in each column of the left table.*

Syntax

Following is the basic syntax of Right Join in SQL −

```
SELECT table1.column1, table2.column2...
FROM table1
RIGHT JOIN table2
ON table1.common_field = table2.common_field;
```

Example

The tables we are using in this example are named CUSTOMERS and ORDERS.

Assume we are creating a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary etc.

Open Compiler

```
CREATE TABLE CUSTOMERS (
```

```
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now, insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |

| | | | | |
|---|---|---|---|---|
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Let us create another table ORDERS, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
  OID INT NOT NULL,
  DATE VARCHAR (20) NOT NULL,
  CUSTOMER_ID INT NOT NULL,
  AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |

| | | | |
|---|---|---|---|
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Now, let us join these two tables using the Right Join query as follows −

Open Compiler

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
RIGHT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Output

This would produce the following result −

| ID | NAME | AMOUNT | DATE |
|---|---|---|---|
| 3 | Kaushik | 3000.00 | 2009-10-08 00:00:00 |
| 3 | Kaushik | 1500.00 | 2009-10-08 00:00:00 |
| 2 | Khilan | 1560.00 | 2009-11-20 00:00:00 |
| 4 | Chaitali | 2060.00 | 2008-05-20 00:00:00 |

## Joining Multiple Tables with Right Join

Like Left Join, Right Join also joins multiple tables. However, the contrast occurs where the second table is returned as a whole instead of the first.

In addition, the rows of first table are matched with the rows in second table. If the records are not matched and the number of records in the second table is greater than the first, NULL is returned as the values in first table.

Syntax

Following is the syntax to join multiple tables using Right Join −

```
SELECT column1, column2, column3...
FROM table1
RIGHT JOIN table2
ON condition_1
RIGHT JOIN table3
ON condition_2
....
....
RIGHT JOIN tableN
ON condition_N;
```

Example

Here, let us consider the previously created tables CUSTOMERS and ORDERS; and create a new table named EMPLOYEE using the following query −

Open Compiler

```
CREATE TABLE EMPLOYEE (
   EID INT NOT NULL,
   EMPLOYEE_NAME VARCHAR (30) NOT NULL,
   SALES_MADE DECIMAL (20)
);
```

Now, we can insert values into this empty tables using the INSERT statement as follows −

Open Compiler

```
INSERT INTO EMPLOYEE VALUES
```

(102, 'SARIKA', 4500),

(100, 'ALEKHYA', 3623),

(101, 'REVATHI', 1291),

(103, 'VIVEK', 3426);

The details of EMPLOYEE table can be seen below −

| EID | EMPLOYEE_NAME | SALES_MADE |
|---|---|---|
| 102 | SARIKA | 4500 |
| 100 | ALEKHYA | 3623 |
| 101 | REVATHI | 1291 |
| 103 | VIVEK | 3426 |

Following query joins these three tables using the **Right Join query** −

Open Compiler

```
SELECT CUSTOMERS.ID, CUSTOMERS.NAME,
ORDERS.DATE, EMPLOYEE.EMPLOYEE_NAME
FROM CUSTOMERS
RIGHT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
RIGHT JOIN EMPLOYEE
ON ORDERS.OID = EMPLOYEE.EID;
```

Through this query, we will display the id, name of the customer along with the date on which the orders are made and the name of the employee who sold the item.

Output

The resultant table is obtained as follows −

| ID | NAME | DATE | EMPLOYEE_NAME |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 3 | Kaushik | 2009-10-08 00:00:00 | SARIKA |
| 3 | Kaushik | 2009-10-08 00:00:00 | ALEKHYA |
| 2 | Khilan | 2009-11-20 00:00:00 | REVATHI |
| 4 | Chaitali | 2008-05-20 00:00:00 | VIVEK |

## Right Join with WHERE Clause

A WHERE Clause is used to filter out records that satisfy the condition specified by it. This clause can be used with the Right Join query to apply certain filters on the joined result-set.

Syntax

The syntax of Right Join when used with WHERE clause is given below −

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

Example

Records in the combined database tables can be filtered using the WHERE clause. Consider the previous two tables CUSTOMERS and ORDERS; and join them using the right join query by applying some constraints using the WHERE clause.

Open Compiler

```
SELECT ID, NAME, DATE, AMOUNT FROM CUSTOMERS
RIGHT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
WHERE ORDERS.AMOUNT > 1000.00;
```

Output

The resultant table after applying the where clause with right join contains the rows that has amount values greater than 1000.00 −

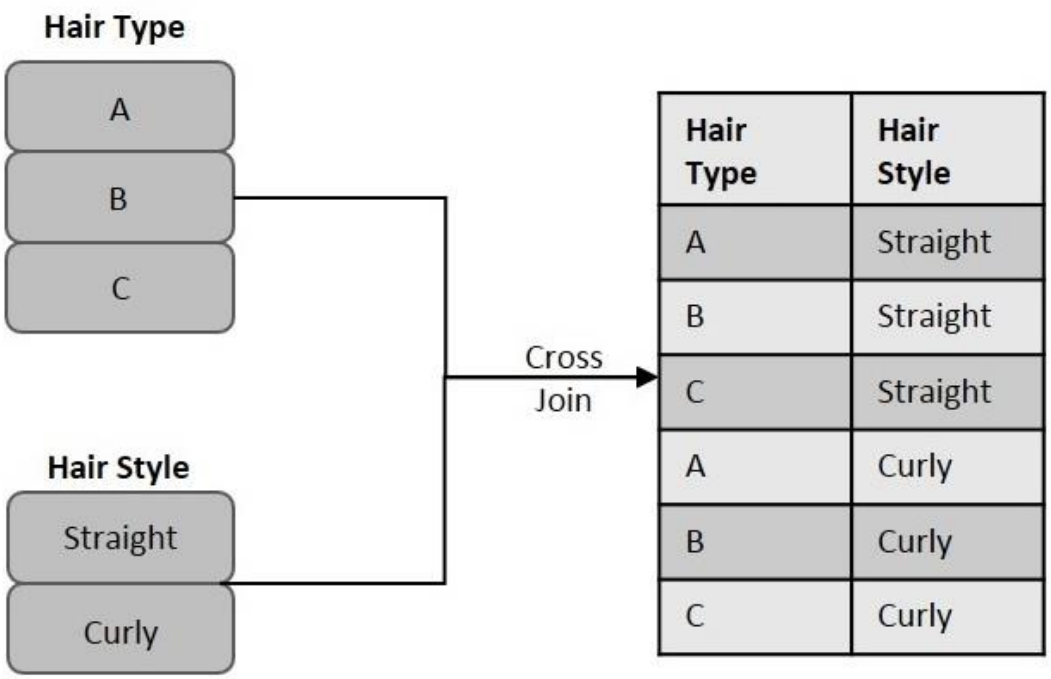
| ID | NAME | DATE | Amount |
|---|---|---|---|
| 3 | Kaushik | 2009-10-08 00:00:00 | 3000.00 |
| 3 | Kaushik | 2009-10-08 00:00:00 | 1500.00 |
| 2 | Khilan | 2009-11-20 00:00:00 | 1560.00 |
| 4 | Chaitali | 2008-05-20 00:00:00 | 2060.00 |

## The SQL Cross Join

An **SQL Cross Join** is a basic type of inner join that is used to retrieve the Cartesian product (or cross product) of two individual tables. That means, this join will combine each row of the first table with each row of second table (i.e. permutations).

*A Cartesian product, or a cross product, is the result achieved from multiplication of two sets. This is done by multiplying all the possible pairs from both the sets.*

The sample figure below illustrates the cross join in a simple manner.




As you can see, we considered two table columns: Hair Style and Hair Type. Each of these columns contain some records that need to be matched. Hence, using cross join, we combine each record in the "Hair Style" column with all records in the "Hair Type" column. The resultant table obtained is considered as the Cartesian product or Joined table.

Syntax

Following is the basic syntax of the Cross Join query in SQL −

```
SELECT column_name(s)
FROM table1
CROSS JOIN table2;
```

Example

Assume we have created a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary etc., using the following query −

Open Compiler

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
```

```
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now, insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |

Let us create another table ORDERS, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
  OID INT NOT NULL,
  DATE VARCHAR (20) NOT NULL,
  CUSTOMER_ID INT NOT NULL,
  AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
```

(100, '2009-10-08 00:00:00', 3, 1500.00),

(101, '2009-11-20 00:00:00', 2, 1560.00);

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |

Now, if we execute the following **Cross Join query** on these two tables given above, the cross join combines each row in CUSTOMERS table with each row in ORDERS table.

Open Compiler

```
SELECT  ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
CROSS JOIN ORDERS;
```

Output

The resultant table is as follows −

| ID | NAME | AMOUNT | DATE |
|---|---|---|---|
| 2 | Khilan | 1500.00 | 2009-10-08 00:00:00 |
| 1 | Ramesh | 1560 | 2009-11-20 00:00:00 |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 |
| 1 | Ramesh | 1500.00 | 2009-10-08 00:00:00 |

## Joining Multiple Tables with Cross Join

We can also join more than two tables using cross join. In this case, multiple-way permutations are displayed and the resultant table is expected to contain way more records than the individual tables.

Syntax

Following is the syntax to join multiple tables using cross join in SQL –

```
SELECT column_name(s)
FROM table1
CROSS JOIN table2
CROSS JOIN table3
CROSS JOIN table4
....
....
....
CROSS JOIN tableN;
```

Example

Assume we have created another table named ORDER_RANGE using the following query −

Open Compiler

```
CREATE TABLE ORDER_RANGE (
   SNO INT NOT NULL,
   ORDER_RANGE VARCHAR (20) NOT NULL
);
```

Now, we can insert values into this empty tables using the INSERT statement as follows −

Open Compiler

```
INSERT INTO ORDER_RANGE VALUES
(1, '1-100'),
(2, '100-200'),
(3, '200-300');
```

The ORDER_RANGE table is created as follows −

| SNO | ORDER_RANGE |
|---|---|
| 1 | 1-100 |
| 2 | 100-200 |
| 3 | 200-300 |

Following query combines the three tables CUSTOMERS, ORDERS and ORDER_RANGE, using cross join −

Open Compiler

```
SELECT ID, NAME, AMOUNT, DATE, ORDER_RANGE
FROM CUSTOMERS
CROSS JOIN ORDERS
CROSS JOIN ORDER_RANGE;
```

Output

The resultant table is given below −

| ID | NAME | AMOUNT | DATE | ORDER_RANGE |
|---|---|---|---|---|
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 | 1-100 |
| 1 | Ramesh | 1560 | 2009-11-20 00:00:00 | 1-100 |

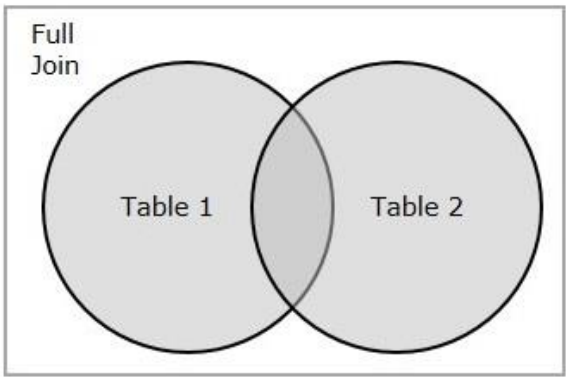
| | | | | |
|---|---|---|---|---|
| 2 | Khilan | 1500.00 | 2009-10-08 00:00:00 | 1-100 |
| 1 | Ramesh | 1500.00 | 2009-10-08 00:00:00 | 1-100 |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 | 100-200 |
| 1 | Ramesh | 1560 | 2009-11-20 00:00:00 | 100-200 |
| 2 | Khilan | 1500.00 | 2009-10-08 00:00:00 | 100-200 |
| 1 | Ramesh | 1500.00 | 2009-10-08 00:00:00 | 100-200 |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 | 200-300 |
| 1 | Ramesh | 1560 | 2009-11-20 00:00:00 | 200-300 |
| 2 | Khilan | 1500.00 | 2009-10-08 00:00:00 | 200-300 |
| 1 | Ramesh | 1500.00 | 2009-10-08 00:00:00 | 200-300 |

## The SQL Full Join

**SQL Full Join** creates a new table by joining two tables as a whole. The joined table contains all records from both the tables and fills NULL values for missing matches on either side. In short, full join is a type of outer join that combines the result-sets of both left and right joins.

*MySQL does not support Full Outer Join. Instead, you can imitate its working by performing union operation between the result-sets obtained from Left Join and Right Join.*

Let us understand this concept in detail with the help of a Venn diagram below. Assume that we have two tables as two sets (represented by circles). The result-set (or newly joined table) obtained using full join is nothing but the union of these two sets.




*You can also achieve the equivalent result-set of FULL JOIN by performing the UNION operation on result-sets of the LEFT JOIN and RIGHT JOIN.*

Syntax

Following is the basic syntax of Full Join in SQL −

```
SELECT column_name(s)
FROM table1
FULL JOIN table2
ON table1.column_name = table2.column_name;
```

Example

Assume we have created a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary etc.Using the following query −

Open Compiler

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
```

```
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |

| | | | | |
|---|---|---|---|---|
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Let us create another table ORDERS, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
  OID INT NOT NULL,
  DATE VARCHAR (20) NOT NULL,
  CUSTOMER_ID INT NOT NULL,
  AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Following query joins the two tables CUSTOMERS and ORDERS in SQL Server −

Open Compiler

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
FULL JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Output

The resultant table is produced as follows −

| ID | NAME | AMOUNT | DATE |
|---|---|---|---|
| 1 | Ramesh | NULL | NULL |
| 2 | Khilan | 1560 | 2009-11-20 00:00:00 |
| 3 | Kaushik | 3000 | 2009-10-08 00:00:00 |
| 3 | Kaushik | 1500 | 2009-10-08 00:00:00 |
| 4 | Chaitali | 2060 | 2008-05-20 00:00:00 |
| 5 | Hardik | NULL | NULL |
| 6 | Komal | NULL | NULL |
| 7 | Muffy | NULL | NULL |

## Joining Multiple Tables with Full Join

The Full Join query can also be used to join more than just two tables. To do that, we sequentially combine two tables at a time, until all the tables are joined together.

Note that in MySQL database, there is no provision to directly use the FULL JOIN keyword to perform join operation on multiple tables. Instead,

calculate the UNION of LEFT JOIN and RIGHT JOIN on two tables at a time, until all the tables are joined.

Syntax

The syntax to join multiple tables using Full Join is given below −

```
SELECT column1, column2, column3...
FROM table1
FULL JOIN table2
ON condition_1
FULL JOIN table3
ON condition_2
....
....
FULL JOIN tableN
ON condition_N;
```

Example

To demonstrate Full Join, let us consider the sample tables CUSTOMERS and ORDERS that we previously created, and create another table name EMPLOYEE using the following query −

Open Compiler

```
CREATE TABLE EMPLOYEE (
   EID INT NOT NULL,
   EMPLOYEE_NAME VARCHAR (30) NOT NULL,
   SALES_MADE DECIMAL (20)
);
```

Now, we can insert values into this empty tables using the INSERT statement as follows −

Open Compiler

INSERT INTO EMPLOYEE VALUES

(102, 'SARIKA', 4500),

(100, 'ALEKHYA', 3623),

(101, 'REVATHI', 1291),

(103, 'VIVEK', 3426);

The EMPLOYEE table created, will be as shown below −

| EID | EMPLOYEE_NAMENAME | SALES_MADE |
|---|---|---|
| 102 | SARIKA | 4500 |
| 100 | ALEKHYA | 3623 |
| 101 | REVATHI | 1291 |
| 103 | VIVEK | 3426 |

Let us join these three tables using the full join query given below −

SELECT CUSTOMERS.ID, CUSTOMERS.NAME, ORDERS.DATE, EMPLOYEE.EMPLOYEE_NAME

FROM CUSTOMERS

FULL JOIN ORDERS

ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID

FULL JOIN EMPLOYEE

ON ORDERS.OID = EMPLOYEE.EID;

Through this query, we will display the id, name of the customer along with the date on which the orders are made and the name of the employee who sold the item.

Output

The resultant table is obtained as follows −

| ID | NAME | DATE | EMPLOYEE_NAME |
|---|---|---|---|
| 1 | Ramesh | NULL | NULL |
| 2 | Khilan | 2009-11-20 00:00:00 | REVATHI |
| 3 | Kaushik | 2009-10-08 00:00:00 | ALEKHYA |
| 3 | Kaushik | 2009-10-08 00:00:00 | SARIKA |
| 4 | Chaitali | 2008-05-20 00:00:00 | VIVEK |
| 5 | Hardik | NULL | NULL |
| 6 | Komal | NULL | NULL |
| 7 | Muffy | NULL | NULL |

## Full Join with WHERE Clause

Joins use the ON clause to filter records by default. Let us suppose there is a further requirement to filter these records based on a certain condition/constraint, we can also make use of the WHERE clause with Joins.

Syntax

The syntax of Full Join when used with WHERE clause is given below −

```
SELECT column_name(s)
FROM table1
FULL JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

Example

Consider the previous two tables CUSTOMERS and ORDERS, and join them using the following Full Join query by applying some constraints using the WHERE clause.

Open Compiler

```
SELECT ID, NAME, DATE, AMOUNT FROM CUSTOMERS
FULL JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
WHERE ORDERS.AMOUNT > 2000.00;
```

Output

The resultant table after applying the WHERE clause with full join contains the rows that has amount values greater than 2000.00 −

| ID | NAME | DATE | AMOUNT |
|---|---|---|---|
| 3 | Kaushik | 2009-10-08 00:00:00 | 3000.00 |
| 4 | Chaitali | 2008-05-20 00:00:00 | 2060.00 |

## SQL - Self Join

**Self Join**, as its name suggests, is a type of join that combines the records of a table with itself.

Suppose an organization, while organizing a Christmas party, is choosing a Secret Santa among its employees based on some colors. It is designed to be done by assigning one color to each of its employees and having them pick a color from the pool of various colors. In the end, they will become the Secret Santa of an employee this color is assigned to.

As we can see in the figure below, the information regarding the colors assigned and a color each employee picked is entered into a table. The table is joined to itself using self join over the color columns to match employees with their Secret Santa.

| Color | Name | Color |
|---|---|---|
| Blue | John | Red |
| Green | Alex | Blue |
| Red | Simon | Green |

| Color | Name | Color |
|---|---|---|
| Blue | John | Red |
| Green | Alex | Blue |
| Red | Simon | Green |

| Name | Secret_Santa |
|---|---|
| John | Simon |
| Alex | John |
| Simon | Alex |

## The SQL Self Join

The **SQL Self Join** is used to join a table to itself as if the table were two tables. To carry this out, alias of the tables should be used at least once.

Self Join is a type of inner join, which is performed in cases where the comparison between two columns of a same table is required; probably to establish a relationship between them. In other words, a table is joined with itself when it contains both **Foreign Key** and **Primary Key** in it.

*Unlike queries of other joins, we use WHERE clause to specify the condition for the table to combine with itself; instead of the ON clause.*

Syntax

Following is the basic syntax of SQL Self Join −

```
SELECT column_name(s)
FROM table1 a, table1 b
WHERE a.common_field = b.common_field;
```

Here, the WHERE clause could be any given expression based on your requirement.

Example

Self Join only requires one table, so, let us create a CUSTOMERS table containing the customer details like their names, age, address and the salary they earn.

Open Compiler

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

Now, insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |

| | | | | |
|---|---|---|---|---|
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Now, let us join this table using the following Self Join query. Our aim is to establish a relationship among the said Customers on the basis of their earnings. We are doing this with the help of the WHERE clause.

Open Compiler

```
SELECT a.ID, b.NAME as EARNS_HIGHER, a.NAME
as EARNS_LESS, a.SALARY as LOWER_SALARY
FROM CUSTOMERS a, CUSTOMERS b
WHERE a.SALARY < b.SALARY;
```

Output

The resultant table displayed will list out all the customers that earn lesser than other customers −

| ID | EARNS_HIGHER | EARNS_LESS | LOWER_SALARY |
|---|---|---|---|
| 2 | Ramesh | Khilan | 1500.00 |
| 2 | Kaushik | Khilan | 1500.00 |
| 6 | Chaitali | Komal | 4500.00 |
| 3 | Chaitali | Kaushik | 2000.00 |
| 2 | Chaitali | Khilan | 1500.00 |
| 1 | Chaitali | Ramesh | 2000.00 |

| | | | |
|---|---|---|---|
| 6 | Hardik | Komal | 4500.00 |
| 4 | Hardik | Chaitali | 6500.00 |
| 3 | Hardik | Kaushik | 2000.00 |
| 2 | Hardik | Khilan | 1500.00 |
| 1 | Hardik | Ramesh | 2000.00 |
| 3 | Komal | Kaushik | 2000.00 |
| 2 | Komal | Khilan | 1500.00 |
| 1 | Komal | Ramesh | 2000.00 |
| 6 | Muffy | Komal | 4500.00 |
| 5 | Muffy | Hardik | 8500.00 |
| 4 | Muffy | Chaitali | 6500.00 |
| 3 | Muffy | Kaushik | 2000.00 |
| 2 | Muffy | Khilan | 1500.00 |
| 1 | Muffy | Ramesh | 2000.00 |

Advertisement

## Self Join with ORDER BY Clause

After joining a table with itself using self join, the records in the combined table can also be sorted in an order, using the ORDER BY clause.

Syntax

Following is the syntax for it −

```
SELECT column_name(s)
FROM table1 a, table1 b
WHERE a.common_field = b.common_field
ORDER BY column_name;
```

Example

Let us join the CUSTOMERS table with itself using self join on a WHERE clause; then, arrange the records in an ascending order using the ORDER BY clause with respect to a specified column, as shown in the following query.

Open Compiler

```
SELECT  a.ID, b.NAME as EARNS_HIGHER, a.NAME
as EARNS_LESS, a.SALARY as LOWER_SALARY
FROM CUSTOMERS a, CUSTOMERS b
WHERE a.SALARY < b.SALARY
ORDER BY a.SALARY;
```

Output

The resultant table is displayed as follows −

| ID | EARNS_HIGHER | EARNS_LESS | LOWER_SALARY |
|---|---|---|---|
| 2 | Ramesh | Khilan | 1500.00 |
| 2 | Kaushik | Khilan | 1500.00 |

| | | | |
|---|---|---|---|
| 2 | Chaitali | Khilan | 1500.00 |
| 2 | Hardik | Khilan | 1500.00 |
| 2 | Komal | Khilan | 1500.00 |
| 2 | Muffy | Khilan | 1500.00 |
| 3 | Chaitali | Kaushik | 2000.00 |
| 1 | Chaitali | Ramesh | 2000.00 |
| 3 | Hardik | Kaushik | 2000.00 |
| 1 | Hardik | Ramesh | 2000.00 |
| 3 | Komal | Kaushik | 2000.00 |
| 1 | Komal | Ramesh | 2000.00 |
| 3 | Muffy | Kaushik | 2000.00 |
| 1 | Muffy | Ramesh | 2000.00 |
| 6 | Chaitali | Komal | 4500.00 |
| 6 | Hardik | Komal | 4500.00 |
| 6 | Muffy | Komal | 4500.00 |
| 4 | Hardik | Chaitali | 6500.00 |
| 4 | Muffy | Chaitali | 6500.00 |
| 5 | Muffy | Hardik | 8500.00 |

## The SQL DELETE... JOIN Clause

The purpose of Joins in SQL is to combine records of two or more tables based on common columns/fields. Once the tables are joined, performing the deletion operation on the obtained result-set will delete records from all the original tables at a time.

For example, consider a database of an educational institution. It consists of various tables: Departments, StudentDetails, LibraryPasses, LaboratoryPasses etc. When a set of students are graduated, all their details from the organizational tables need to be removed, as they are unwanted. However, removing the details separately from multiple tables can be cumbersome.

To make it simpler, we will first retrieve the combined data of all graduated students from all the tables using **Joins**; then, this joined data is deleted from all the tables using **DELETE** statement. This entire process can be done in one single query.

Syntax

Following is the basic syntax of the SQL DELETE... JOIN statement −

```
DELETE table(s)

FROM table1 JOIN table2

ON table1.common_field = table2.common_field;
```

When we say JOIN here, we can use any type of Join: Regular Join, Natural Join, Inner Join, Outer Join, Left Join, Right Join, Full Join etc.

Example

To demonstrate this deletion operation, we must first create tables and insert values into them. We can create these tables using CREATE TABLE queries as shown below.

Create a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary etc. Using the following query −

Open Compiler

```
CREATE TABLE CUSTOMERS (
```

```
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now, insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |

| | | | | |
|---|---|---|---|---|
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Let us create another table ORDERS, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
  OID INT NOT NULL,
  DATE VARCHAR (20) NOT NULL,
  CUSTOMER_ID INT NOT NULL,
  AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |

| | | | |
|---|---|---|---|
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Following **DELETE... JOIN query** removes records from these tables at once −

```
DELETE a
FROM CUSTOMERS AS a INNER JOIN ORDERS AS b
ON a.ID = b.CUSTOMER_ID;
```

Output

The output will be displayed in SQL as follows −

Query OK, 3 rows affected (0.01 sec)

Verification

We can verify whether the changes are reflected in a table by retrieving its contents using the SELECT statement as follows −

```
SELECT * FROM CUSTOMERS;
```

The table is displayed as follows −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Since, we only deleted records from CUSTOMERS table, the changes will not be reflected in the ORDERS table. We can verify it using the following query.

```
SELECT * FROM ORDERS;
```

The ORDERS table is displayed as −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Fullscreen

## DELETE... JOIN with WHERE Clause

The **ON** clause in DELETE... JOIN query is used to apply constraints on the records. In addition to it, we can also use the WHERE clause to make the filtration stricter. Observe the query below. Here, we are deleting the records of customers, in the **CUSTOMERS** table, whose salary is lower than Rs. 2000.00.

```
DELETE a
FROM CUSTOMERS AS a INNER JOIN ORDERS AS b
ON a.ID = b.CUSTOMER_ID
WHERE a.SALARY < 2000.00;
```

Output

On executing the query, following output is displayed.

Query OK, 1 row affected (0.01 sec)

Verification

We can verify whether the changes are reflected in a table by retrieving its contents using the SELECT statement as follows −

```
SELECT * FROM CUSTOMERS;
```

The CUSTOMERS table after deletion is as follows −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Since we only deleted records from the CUSTOMERS table, the changes will not be reflected in the ORDERS table. We can verify it using the following query −

```
SELECT * FROM ORDERS;
```

The ORDERS table is displayed as −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

## The SQL UPDATE... JOIN Clause

The **UPDATE** statement only modifies the data in a single table and **JOINS** in SQL are used to fetch the combination of rows from multiple tables, with respect to a matching field.

If we want to update data in multiple tables, we can combine multiple tables into one using JOINS and then update them using UPDATE statement. This is also known as cross-table modification.

Syntax

Following is the basic syntax of the SQL UPDATE... JOIN statement −

```
UPDATE table(s)
JOIN table2 ON table1.join_column = table2.join_column
SET table1.column1 = table2.new_value1,
   table1.column2 = table2.new_value2;
```

Where, **JOIN** can be: Regular Join, Natural Join, Inner Join, Outer Join, Left Join, Right Join, Full Join etc.

Example

Assume we have created a table named CUSTOMERS, which contains the personal details of customers including their name, age, address and salary etc., using the following query −

Open Compiler

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now, insert values into this table using the INSERT statement as follows −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

The table will be created as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Let us create another table ORDERS, containing the details of orders made and the date they are made on.

Open Compiler

```
CREATE TABLE ORDERS (
   OID INT NOT NULL,
   DATE VARCHAR (20) NOT NULL,
```

```
  CUSTOMER_ID INT NOT NULL,
  AMOUNT DECIMAL (18, 2)
);
```

Using the INSERT statement, insert values into this table as follows −

Open Compiler

```
INSERT INTO ORDERS VALUES
(102, '2009-10-08 00:00:00', 3, 3000.00),
(100, '2009-10-08 00:00:00', 3, 1500.00),
(101, '2009-11-20 00:00:00', 2, 1560.00),
(103, '2008-05-20 00:00:00', 4, 2060.00);
```

The table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3000.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 1500.00 |
| 101 | 2009-11-20 00:00:00 | 2 | 1560.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2060.00 |

Following UPDATE... JOIN query increments the salary of customers by 1000 with respect to the inflation of their order amount by 500 –

```
UPDATE CUSTOMERS
JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
```

SET CUSTOMERS.SALARY = CUSTOMERS.SALARY + 1000,

ORDERS.AMOUNT = ORDERS.AMOUNT + 500;

Verification

We can verify whether the changes are reflected in a table by retrieving its contents using the SELECT statement as follows −

SELECT * FROM CUSTOMERS;

The updated CUSTOMERS table is displayed as follows −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 2500.00 |
| 3 | Kaushik | 23 | Kota | 3000.00 |
| 4 | Chaitali | 25 | Mumbai | 7500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

Now, check whether the ORDERS table is updated using the following SELECT statement −

SELECT * FROM ORDERS;

The updated ORDERS table is displayed as follows −

| OID | DATE | CUSTOMER_ID | AMOUNT |
|---|---|---|---|
| 102 | 2009-10-08 00:00:00 | 3 | 3500.00 |
| 100 | 2009-10-08 00:00:00 | 3 | 2000.00 |

| | | | |
|---|---|---|---|
| 101 | 2009-11-20 00:00:00 | 2 | 2060.00 |
| 103 | 2008-05-20 00:00:00 | 4 | 2560.00 |

## UPDATE... JOIN with WHERE Clause

While updating records from multiple tables, if we use the WHERE clause along with the **UPDATE... JOIN** statement we can filter the records to be updated (from the combined result set).

### Syntax

The syntax of SQL UPDATE... JOIN with WHERE clause in MySQL database is as follows −

```
UPDATE table(s)
JOIN table2 ON column3 = column4
SET table1.column1 = value1, table1.column2 = value2, ...
WHERE condition;
```

Example

Now, let us execute the following query to increase the salary of customer whose id is **3** −

```
UPDATE CUSTOMERS
LEFT JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID
SET CUSTOMERS.SALARY = CUSTOMERS.SALARY + 1000
WHERE ORDERS.CUSTOMER_ID = 3;
```

Verification

We can verify whether the changes are reflected in a table by retrieving its contents using the SELECT statement as follows.

```
SELECT * FROM CUSTOMERS;
```

As we can see in the table below, SALARY value of "Kaushik" is increased by 1000 −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 1500.00 |
| 3 | Kaushik | 23 | Kota | 3000.00 |
| 4 | Chaitali | 25 | Mumbai | 6500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

## The UPDATE... JOIN Clause in SQL Server

The SQL UPDATE... JOIN Clause also works in SQL Server database. But, the syntax of the query is slightly different from that of MySQL. However, the working of it is exactly the same as MySQL query.

*In MySQL, the UPDATE statement is followed by the JOIN clause and SET statements respectively. Whereas, in MS SQL Server the SET statement is followed by the JOIN clause.*

Syntax

Following is the syntax of the **UPDATE... JOIN** in SQL Server −

```
UPDATE tables(s)
SET column1 = value1, column2 = value2, ...
FROM table1
JOIN table2 ON table1.join_column = table2.join_column;
```

Example

In this example, we will update values of the CUSTOMERS and ORDERS table that we created above; using the following UPDATE... JOIN query −

```
UPDATE CUSTOMERS
SET SALARY = SALARY + 1000
FROM CUSTOMERS
JOIN ORDERS ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Verification

We can verify whether the changes are reflected in a table by retrieving its contents using the SELECT statement as follows.

```
SELECT * FROM CUSTOMERS;
```

The updated CUSTOMERS table is displayed as follows −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Delhi | 2500.00 |
| 3 | Kaushik | 23 | Kota | 3000.00 |
| 4 | Chaitali | 25 | Mumbai | 7500.00 |
| 5 | Hardik | 27 | Bhopal | 8500.00 |
| 6 | Komal | 22 | Hyderabad | 4500.00 |
| 7 | Muffy | 24 | Indore | 10000.00 |

## Working of Left Join

**Left Join** or **Left Outer Join** in SQL combines two or more tables, where the first table is returned as it is; but, only the record(s) that have counterparts in first table are returned from consequent tables.

If the ON clause matches zero records in consequent tables with the rows in first table, left join will still return these rows of first table in the result, but with NULL in each column from the right table.

Syntax

Following is the basic syntax of Left Join in SQL −

```
SELECT table1.column1, table2.column2...
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Example

The example below demonstrates the Left Join operation on two relative tables. Here, the first table contains the salary information while the second table contains marital status information. Since Alex's status is unknown, it is not recorded in the table.

| ID | Name | Salary |
|---|---|---|
| 1 | John | 40000 |
| 2 | Alex | 25000 |
| 3 | Simon | 43000 |

+

| ID | Name | Status |
|---|---|---|
| 1 | John | Married |
| 3 | Simon | Married |

=

| ID | Name | Salary | Status |
|---|---|---|---|
| 1 | John | 40000 | Married |
| 2 | Alex | 25000 | NULL |
| 3 | Simon | 43000 | Married |

When both tables are joined using the Left Join query, since there is no record matching Alex's Status, the value is recorded as NULL in the final table.

Working of Right Join

Right Join or Right Outer Join query in SQL returns all rows from the right table, even if there are no matches in the left table. This means that if the ON clause matches 0 (zero) records in left table with the records in right table; right join will still return the rows of right table in the result, but with a NULL value in each column of the left table.

Syntax

Following is the basic syntax of a Right Join in SQL −

```
SELECT table1.column1, table2.column2...
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

Example

Now in this example, the Right Join operation is performed on the same tables. Here, we are starting the join from the right table; since, the right table does not contain the record value matching Alex's row, the row is discarded from the final table.

| ID | Name | Salary |
|---|---|---|
| 1 | John | 40000 |
| 2 | Alex | 25000 |
| 3 | Simon | 43000 |

+

| ID | Name | Status |
|---|---|---|
| 1 | John | Married |
| 3 | Simon | Married |

=

| ID | Name | Status | Salary |
|---|---|---|---|
| 1 | John | Married | 40000 |
| 3 | Simon | Married | 43000 |

The final table only consists of two rows as the right table consists of two rows only.

## Left Join Vs Right Join

Let us summarize all the differences between the Left Join and Right Join in the table below −

| **Left Join** | **Right Join** |
|---|---|
| Left Join matches the data of the first table or the left table with the data in second table. If the data is matched, the records are combined; otherwise, NULL is recorded. | Right Join matches the data of the second table or right table with the data in first table. If the data is matched, the records are combined; otherwise, NULL is recorded. |
| If the first table has less rows than the second table, extra unmatched rows from the second table are discarded. | If the second table has less rows than the first table, extra unmatched rows from the first table are discarded. |
| This Join is also known as Left Outer Join | This Join is also known as Right Outer Join |
| *= is used in Transact SQL, instead of using the LEFT JOIN or LEFT OUTER JOIN query. | =* is used in Transact SQL, instead of using the RIGHT JOIN or RIGHT OUTER JOIN query. |

Working of UNION

UNION is a type of operator/clause in SQL, that works similar to the union operator in relational algebra. It does nothing more than just combining information from multiple tables that are union compatible.

The tables are said to be union compatible if they follow the conditions given below −

- The tables to be combined must have same number of columns with the same datatype.
- The number of rows need not be same.

Once these criteria are met, UNION operator returns all the rows from multiple tables, after eliminating duplicate rows, as a resultant table.

**Note** − Column names of first table will become column names of resultant table, and contents of second table will be merged into resultant columns of same data type.

Syntax

Following is the syntax of the SQL UNION operator −

```
SELECT * FROM table1
UNION
SELECT * FROM table2;
```

Example

Let us first create two table "COURSES_PICKED" and "EXTRA_COURSES_PICKED" with the same number of columns having same data types.

Create table COURSES_PICKED using the following query −

Open Compiler

```
CREATE TABLE COURSES_PICKED(
   STUDENT_ID INT NOT NULL,
   STUDENT_NAME VARCHAR(30) NOT NULL,
   COURSE_NAME VARCHAR(30) NOT NULL
);
```

Insert values into the COURSES_PICKED table with the help of the query given below −

Open Compiler

```
INSERT INTO COURSES_PICKED VALUES
(1, 'JOHN', 'ENGLISH'),
(2, 'ROBERT', 'COMPUTER SCIENCE'),
(3, 'SASHA', 'COMMUNICATIONS'),
(4, 'JULIAN', 'MATHEMATICS');
```

Create table EXTRA_COURSES_PICKED using the following query −

Open Compiler

```
CREATE TABLE EXTRA_COURSES_PICKED(
   STUDENT_ID INT NOT NULL,
   STUDENT_NAME VARCHAR(30) NOT NULL,
   EXTRA_COURSE_NAME VARCHAR(30) NOT NULL
);
```

Following is the query to insert values into the EXTRA_COURSES_PICKED table −

Open Compiler

```
INSERT INTO EXTRA_COURSES_PICKED VALUES
(1, 'JOHN', 'PHYSICAL EDUCATION'),
(2, 'ROBERT', 'GYM'),
(3, 'SASHA', 'FILM'),
(4, 'JULIAN', 'PHOTOGRAPHY');
```

Now, let us combine the tables COURSES_PICKED and EXTRA_COURSES_PICKED, using the UNION query as follows −

Open Compiler

```
SELECT * FROM COURSES_PICKED
UNION
SELECT * FROM EXTRA_COURSES_PICKED;
```

Output

The resultant table obtained after performing the UNION operation is −

| STUDENT_ID | STUDENT_NAME | COURSE_NAME |
|---|---|---|
| 1 | Jhon | English |
| 1 | Jhon | Physical Education |
| 2 | Robert | Computer Science |

| | | |
|---|---|---|
| 2 | Robert | Gym |
| 3 | Shasha | Communications |
| 3 | Shasha | Film |
| 4 | Julian | Mathematics |
| 4 | Julian | Photography |

## UNION vs JOIN

### Working of JOIN

The Join operation is used to combine information from multiple related tables into one, based on their common fields. This operation can be used with various clauses like ON, WHERE, ORDER BY, GROUP BY etc.

There are two types of Joins −

- Inner Join
- Outer Join

The basic type of join is an Inner Join, which only retrieves the matching values of common columns. It is a default join.

The result table of the Outer join includes both matched and unmatched rows from the first table. It is divided into subtypes like Left Join, Right Join, and Full Join.

Syntax

Following is the basic syntax of a Join operation in SQL −

SELECT column_name(s)

FROM table1

JOIN table2

ON table1.column_name = table2.column_name;

Example

In the following example, we will join the same tables we created above, i.e., COURSES_PICKED and EXTRA_COURSES_PICKED, using the query below -

SELECT c.STUDENT_ID, c.STUDENT_NAME, COURSE_NAME, COURSES_PICKED

FROM COURSES_PICKED c

JOIN EXTRA_COURSES_PICKED e

ON c.STUDENT_ID = e.STUDENT_ID;

Output

The resultant table will be displayed as follows −

| STUDENT_ID | STUDENT_NAME | COURSE_NAME | COURSE_PICKED |
|---|---|---|---|
| 1 | Jhon | ENGLISH | Physical Education |
| 2 | Robert | COMPUTER SCIENCE | Gym |
| 3 | Shasha | COMMUNICATIONS | Film |
| 4 | Julian | MATHEMATICS | Photography |

UNION Vs JOIN

As we saw in the examples given above, the UNION operator is only executable on tables that are union compatible, whereas, the JOIN operator joins two tables that need not be compatible but should be related.

Let us summarize all the difference between these queries below −

| UNION | JOIN |
| --- | --- |
| UNION operation is only performed on tables that are union compatible, i.e., the tables must contain same number of columns with same data type. | JOIN operation can be performed on tables that has at least one common field between them. The tables need not be union compatible. |
| The data combined will be added as new rows of the resultant table. | The data combined will be adjoined into the resultant table as new columns. |
| This works as the conjunction operation. | This works as an intersection operation. |
| UNION removes all the duplicate values from the resultant tables. | JOIN retains all the values from both tables even if they're redundant. |
| UNION does not need any additional clause to combine two tables. | JOIN needs an additional clause ON to combine two tables based on a common field. |
| It is mostly used in scenarios like, merging the old employees list in an organization with the new employees list. | This is used in scenarios where merging related tables is necessary. For example, combining tables containing customers list and the orders they made. |

## The SQL Unique Key

The **SQL Unique Key** (or, Unique constraint) does not allow duplicate values in a column of a table. It prevents two records from having same values in a column.

*Unique Key is just an alternative to the Primary Key; as both Unique and Primary Key constraints ensure uniqueness in a column of the table.*

Suppose we have a table named CUSTOMERS to store the customer records in a Bank and if one of the column names is MOBILE_NO then, we can create a UNIQUE constraint on this column to prevent the entry of multiple records with the same mobile number.

Features of Unique Keys

Following is the list of some key features of the Unique Key in an SQL database −

- The unique key is similar to the primary key in a table, but it can accept NULL values, whereas the primary key does not.
- It accepts only one NULL value.
- It cannot have duplicate values.
- It can also be used as a foreign key in another table.
- A table can have more than one Unique column.

# Creating SQL Unique Key

You can create a Unique Key on a database table using the **UNIQUE** keyword in SQL. While creating a database table, specify this SQL keyword along with the column (where this key needs to be defined on).

Syntax

Following is the syntax to create a UNIQUE key constraint on a column in a table −

```
CREATE TABLE table_name(
   column1 datatype UNIQUE KEY,
   column2 datatype,
```

```
   .....
   .....
   columnN datatype
);
```

Example

Using the following SQL query, we are creating a table named CUSTOMERS with five fields ID, NAME, AGE, ADDRESS, and SALARY in it. Here, we are creating a Unique Key on the ID column.

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL UNIQUE KEY,
   NAME VARCHAR(20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2)
);
```

Output

Following is the output of the above SQL statement −

```
Query OK, 0 rows affected (0.03 sec)
```

Verification

Since we have created a UNIQUE constraint on the column named ID, we cannot insert duplicate values in it. Let us verify by inserting the following records with duplicate ID values into the CUSTOMERS table −

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(1, 'Khilan', 25, 'Delhi', 1500.00 );
```

On execution, following error is displayed proving that the UNIQUE constraint is indeed defined on the ID column −

```
ERROR 1062 (23000): Duplicate entry '1' for key 'customers.ID'
```

## Multiple Unique Keys

We can create one or more Unique Keys on one or more columns in an SQL table.

Syntax

Following is the syntax to create unique key constraints on multiple columns in a table −

```
CREATE TABLE table_name(
   column1 datatype UNIQUE KEY,
   column2 datatype UNIQUE KEY,
   .....
   .....
   columnN datatype
);
```

Example

Assume we have created a table with the name CUSTOMERS in the SQL database using CREATE TABLE statement. A Unique key is defined on columns ID and NAME using the UNIQUE keyword as shown below –

```
CREATE TABLE BUYERS (
   ID INT NOT NULL UNIQUE KEY,
   NAME VARCHAR(20) NOT NULL UNIQUE KEY,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2)
);
```

Output

Following is the output of the above SQL statement −

Query OK, 0 rows affected (0.03 sec)

Verification

Since we have created a UNIQUE constraint on the column named ID and NAME, we cannot insert duplicate values in it. Let us verify by inserting duplicate records into the BUYERS table using the following INSERT statement −

```
INSERT INTO BUYERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(1, 'Rajesh', 25, 'Delhi', 1500.00 );
```

Following error is displayed −

ERROR 1062 (23000): Duplicate entry '1' for key 'customers.ID'

In the same way if you try to insert the another record with duplicate value for the column NAME as −

```
INSERT INTO BUYERS VALUES (2, 'Ramesh', 36, 'Chennai', 1700.00 );
```

Following error is generated −

ERROR 1062 (23000): Duplicate entry 'Ramesh' for key 'buyers.NAME'

## Unique Key on an Existing Column

Until now, we have only seen how to define a Unique Key on a column while creating a new table. But, we can also add a unique key on an existing column of a table. This is done using the ALTER TABLE... ADD CONSTRAINT statement.

Syntax

Following is the syntax to create a unique constraint on existing columns of a table −

```
ALTER TABLE table_name ADD CONSTRAINT
UNIQUE_KEY_NAME UNIQUE (column_name);
```

**Note** − Here the UNIQUE_KEY_NAME is just the name of the UNIQUE KEY. It is optional to specify and is used to drop the constraint from the column in a table.

Example

In this example, we add a Unique Key on the ADDRESS column of the existing CUSTOMERS table −

```
ALTER TABLE CUSTOMERS ADD CONSTRAINT
UNIQUE_ADDRESS UNIQUE(ADDRESS);
```

Output

Following is the output of the above statement −

Query OK, 0 rows affected (0.05 sec)

Records: 0  Duplicates: 0  Warnings: 0

## Dropping an SQL Unique Key

If you have already created a unique key on a column, you can drop it whenever it is not needed. To drop the Unique Key from the column of a table, you need to use the ALTER TABLE statement.

Syntax

Following is the SQL query to drop the UNIQUE constraint from the column of a table −

```
ALTER TABLE table_name DROP CONSTRAINT UNIQUE_KEY_NAME;
```

Example

Consider the CUSTOMERS table created above, we have created the UNIQUE constraints on three columns named ID, NAME and ADDRESS; drop the UNIQUE constraints from the column ADDRESS by executing the following SQL query −

```
ALTER TABLE CUSTOMERS DROP CONSTRAINT
UNIQUE_ADDRESS;
```

Output

Following is the output of the above statement −

Query OK, 0 rows affected (0.01 sec)

Records: 0  Duplicates: 0  Warnings: 0

Verification

Now, let us insert two duplicate records of column ADDRESS −

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),

(2, 'Khilan', 25, 'Ahmedabad', 1500.00 );

If you verify the contents of the table, you can observe that both the records have the same ADDRESS as shown below −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 2 | Khilan | 25 | Ahmedabad | 1500.00 |

## The SQL Primary Key

The SQL **Primary Key** is a column (or combination of columns) that uniquely identifies each record in a database table. The Primary Key also speeds up data access and is used to establish a relationship between tables.

*Even though a table can only have one Primary Key, it can be defined on one or more fields. When a primary key is created on multiple fields of a table, it is called a Composite Key.*

Let us say, you are developing an application called "Customer Management System" to handle all the customer data of a member-only resort. This data can include their personal details, assigned member IDs, other details of the membership they opted, etc. And in all the tables created within this database, the member ID is used to distinguish the customers from each other. So, this field will be the Primary Key.

Following is the diagram of a CUSTOMERS table that holds the personal details of the customers. And as we can observe, the primary key is defined on the CUST_ID column. Using this primary key, we can retrieve a unique record of any customer.

Points to Remember

Here are some key points of the PRIMARY KEY −

- It contains only a unique value.
- It can not be null.
- One table can have only one Primary Key.
- A primary key length cannot be more than 900 bytes.

## Creating an SQL Primary Key

While creating a table using the **CREATE TABLE statement**, you can add the primary key constraint on a particular column of the table just by to specifying the name of the column along with the keyword "PRIMARY KEY".

Syntax

Following is the syntax to define a column of a table as a primary key −

```
CREATE TABLE table_name(
   column1 datatype,
   column2 datatype,
   column3 datatype,
   .....
```

```
   columnN datatype,
   PRIMARY KEY(column_name)
);
```

Example

In the following example, we are trying to create a table with the name CUSTOMERS with various fields in an SQL database. While creating the table, we will add the constraint "PRIMARY KEY" on the column named ID.

Open Compiler

```
CREATE TABLE CUSTOMERS (
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

Output

Following is the output of the above SQL statement −

Query OK, 0 rows affected (0.03 sec)

Verification

As we know primary key value must be unique, so you can not insert the record with the same ID. Here, we will verify the constraint created on the ID column, by inserting records with duplicate ID values.

First of all, let's insert a record into the CUSTOMERS table −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(3, 'Kaushik', 23, 'Kota', 2000.00);
```

Now, let's insert one more record with same ID −

Open Compiler

INSERT INTO CUSTOMERS VALUES

(3, 'Chaitali', 25, 'Mumbai', 6500.00);

As we have mentioned above, if any field/column is defined as Primary Key in a database table, two records can not have the same value in that column/field. Therefore, the second insert statement generates the following error −

ERROR 1062 (23000): Duplicate entry '3' for key 'customers.PRIMARY'

Similarly, a primary key column cannot contain null values. Here, using the INSERT statement we are passing a NULL value to the primary key column (ID).

Open Compiler

INSERT INTO CUSTOMERS VALUES

(NULL, 'Komal', 22, 'Hyderabad', 4500.00);

This statement generates the following error −

ERROR 1048 (23000): Column 'ID' cannot be null

## Creating Primary Key on an Existing Column

We can also add the PRIMARY KEY constraint on an existing column of a table using the **ALTER TABLE** statement.

Syntax

Following is the syntax to create a primary constraint on existing columns of a table −

ALTER TABLE table_name ADD CONSTRAINT PRIMARY KEY (column_name);

Example

In this example, we are adding the PRIMARY KEY constraint on the NAME column of the existing CUSTOMERS table −

ALTER TABLE CUSTOMERS ADD CONSTRAINT PRIMARY KEY(NAME);

Output

Following is the output of the above statement −

Query OK, 0 rows affected (0.05 sec)

Records: 0  Duplicates: 0  Warnings: 0

## *Dropping an SQL Primary Key*

If you can add a Primary Key Constraint to a column in the table, you can drop it as well. This is done by using the ALTER TABLE... DROP statement.

Syntax

Following is the syntax of the ALTER TABLE statement to can drop the Primary key constraints from the column of a table −

ALTER TABLE table_name DROP PRIMARY KEY;

Example

Let us consider the CUSTOMERS table where we have created a primary key constraint on a column named ID. You can drop this constraint from the column ID by executing the following statement −

ALTER TABLE CUSTOMERS DROP PRIMARY KEY;

Output

The above SQL query produces the following output −

Query OK, 1 row affected (0.03 sec)

Records: 1  Duplicates: 0  Warnings: 0

Verification

As we have dropped the Primary key from the column named ID, we can insert multiple records with the same ID. Following statement inserts four records with the same ID −

INSERT INTO CUSTOMERS VALUES

(3, 'Chaitali', 25, 'Mumbai', 6500.00 ),

(3, 'Hardik', 27, 'Bhopal', 8500.00 ),

(3, 'Komal', 22, 'Hyderabad', 4500.00 ),

(3, 'Muffy', 24, 'Indore', 10000.00 );

If you verify the content of this table, you can find multiple records with same ID −

SELECT * FROM CUSTOMERS;

The table will be displayed as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 3 | Kaushik | 23 | Kota | 2000.00 |
| 3 | Chaitali | 25 | Mumbai | 6500.00 |
| 3 | Hardik | 27 | Bhopal | 8500.00 |
| 3 | Komal | 22 | Hyderabad | 4500.00 |
| 3 | Muffy | 24 | Indore | 10000.00 |

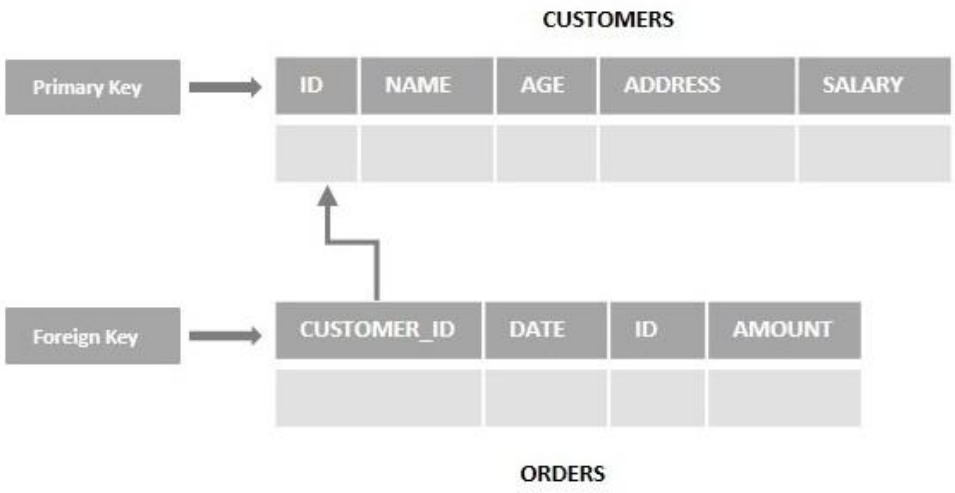
## The SQL Foreign Key

In SQL, a **Foreign Key** is a column in one table that matches a Primary Key in another table, allowing the two tables to be connected together.

A foreign key also maintains referential integrity between two tables, making it impossible to drop the table containing the primary key (preserving the connection between the tables).

*The foreign key can reference the unique fields of any table in the database. The table that has the primary key is known as the parent table and the key with the foreign key is known as the child table.*

Let's consider an example scenario, assume we have two tables namely CUSTOMERS (ID, NAME, AGE, ADDRES, SALARY) and ORDERS (ID, DATE, CUSTOMER_ID, AMOUNT). Here the id of the customer is primary key (ID) in the CUSTOMERS table and foreign key in the ORDERS (CUSTOMER_ID) table observe the following diagram −




Features of Foreign Key

Following is the of features of Foreign Key −

- A Foreign Key is used to reduce the redundancy (or duplicates) in the table.
- It helps to normalize (or organize the data in a database) the data in multiple tables.

Syntax

Following is the basic syntax to add Foreign Key constraints on a column of a table in MySQL database −

```
CREATE TABLE table_name (
   column1 datatype,
   column2 datatype,
   ...
   CONSTRAINT fk_name
        FOREIGN KEY (column_name)
        REFERENCES referenced_table(referenced_column)
);
```

Example

Let us create two tables with the names CUSTOMERS and ORDERS. The following query creates a table with the name CUSTOMERS −

Open Compiler

```
CREATE TABLE CUSTOMERS(
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25) ,
   SALARY DECIMAL (18, 2),
   PRIMARY KEY (ID)
);
```

Output

Following is the output of the above SQL statement −

Query OK, 0 rows affected (0.02 sec)

Now, let us create the ORDERS table. While doing so, we add the foreign key constraint on column CUSTOMER_ID reference on column ID of the CUSTOMERS table as shown in the statement below −

```
CREATE TABLE ORDERS (
   ID INT NOT NULL,
   DATE DATETIME,
   CUSTOMER_ID INT,
   CONSTRAINT FK_CUSTOMER
   FOREIGN KEY(CUSTOMER_ID)
   REFERENCES CUSTOMERS(ID),
   AMOUNT DECIMAL,
   PRIMARY KEY (ID)
);
```

Output

The above statement produces the following output −

Query OK, 0 rows affected (0.04 sec)

Verification

We have created a Foreign Key Constraint on a column named CUSTOMER_ID in the ORDERS table that references the column named ID of the CUSTOMERS table; so you can't drop table1 (CUSTOMERS) before dropping the table2 (ORDERS).

First of all, let's drop the CUSTOMERS table without dropping the ORDERS table using the DROP TABLE statement –

**DROP TABLE CUSTOMERS;**

If you verify the error message below, you will observe that it says that the table can not be dropped because it is referenced by a FOREIGN KEY constraint −

ERROR 3730 (HY000): Cannot drop table 'customers' referenced by a foreign key constraint 'FK_CUSTOMER' on table 'orders'.

Advertisement

## Foreign Key Constraint on an Existing Column

We can also create a Foreign key constraint on a column of an existing table. This is useful when you forget to add a Foreign Key constraint on a column while creating a table, or when you want to add this constraint on another column even if one Foreign Key column exists in a table.

Syntax

Using the ALTER TABLE statement we can add a Foreign Key constraint on an existing column in a table in MySQL database as shown below −

ALTER TABLE TABLE2

ADD CONSTRAINT[symbol]

FOREIGN KEY(column_name)

REFERENCES TABLE1(column_name);

Here, FK_ORDERS is the name of the foreign key constraint. It is optional to specify the name of a constraint but it comes in handy while dropping the constraint.

Example

Assume the CUSTOMERS and ORDERS tables have already been created in the SQL database. Now, we will add a Foreign Key Constraint on the ID column of the ORDERS table.

Following is the SQL query to add the foreign key constraint on an the column of an existing table –

ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS

FOREIGN KEY(ID) REFERENCES CUSTOMERS(ID);

Output

Following is the output of the above program −

Query OK, 0 rows affected (0.04 sec)

Records: 0  Duplicates: 0  Warnings: 0

Verification

We have created a Foreign Key Constraint on a column named CUSTOMER_ID in the ORDERS table that references the column name ID of the CUSTOMERS table. So, you can't drop table1 (CUSTOMERS) before dropping the table2 (ORDERS).

First of all, let us drop the CUSTOMERS table without dropping the ORDERS table by executing the following statement −

DROP TABLE CUSTOMERS;

This generates an error message saying that the table can not be dropped because it is referenced by a FOREIGN KEY constraint −

ERROR 3730 (HY000): Cannot drop table 'customers' referenced by a foreign key constraint 'FK_CUSTOMER' on table 'orders'.

## Dropping a FOREIGN KEY

You can drop the foreign key from a table, without dropping that entire table, using the ALTER TABLE statement.

Syntax

Following is the syntax to drop the FOREIGN key constraint from the column of the table using the ALTER TABLE statement−

ALTER TABLE table_name DROP FOREIGN KEY (constraint symbol);

Where, FK_NAME is the name of the foreign key constraint you need to drop.

Example

The SQL query to drop the foreign key constraint from the column of a table is as follows −

ALTER TABLE ORDERS DROP FOREIGN KEY FK_ORDERS;

Output

Following is the output of the above SQL query −

Query OK, 0 rows affected (0.01 sec)

Records: 0  Duplicates: 0  Warnings: 0

Verification

Since we have dropped the Foreign key constraint from the ORDERS table, you can now directly drop the CUSTOMERS table without dropping the ORDERS table, as shown below −

DROP TABLE CUSTOMERS;

If you verify the below status code thrown by the above SQL command, you observe that the CUSTOMERS table has dropped.

Query OK, 0 rows affected (0.02 sec)

Primary Key vs Foreign Key

Even though both the primary key and foreign key refer to the same column, there are many differences to be observed in the way they work. They are listed below −

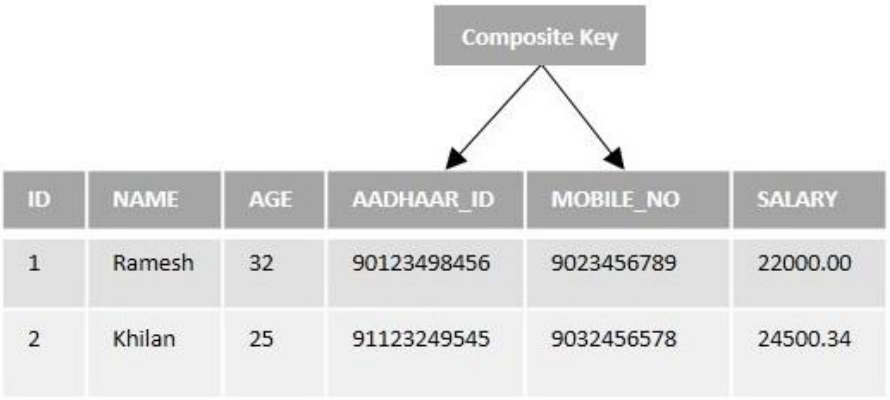
| Primary Key | Foreign Key |
|---|---|
| The primary key is always unique. | The foreign key can be duplicated. |
| The primary key can not be NULL. | The Foreign can be NULL. |
| A table can contain only one Primary Key. | We can have more than one Foreign Key per table. |

## The SQL Composite Key

An SQL **Composite Key** is a key that can be defined on two or more columns in a table to uniquely identify any record. It can also be described as a Primary Key created on multiple columns.

Composite Keys are necessary in scenarios where a database table does not have a single column that can uniquely identify each row from the table. In such cases, we might need to use the combination of columns to ensure that each record in the table is distinct and identifiable.

Let us understand the composite keys with an example. Suppose if we have a table named CUSTOMERS with various fields like ID, NAME, AGE, AADHAAR_ID, MOBILE_NO and SALARY as shown below −




We can select the two columns AADHAAR_ID and MOBILE_NO and define a Composite key on them, and it can be used to fetch the records of the CUSTOMERS table uniquely.

Features of Composite Keys

Following are some important features of the SQL Composite Key −

- A Composite Key can be created by combining more than one Candidate Key.
- Each Candidate Key (or column) that makes up a Composite Key may or may not be a Foreign Key. However, if all the columns of the Composite Key are Foreign Keys in their own right, then the Composite Key is known as a Compound Key.
- A Composite Key cannot be NULL; i.e. any column of the Composite Key must not contain NULL values.
- The individual columns making up the Composite Key can contain duplicate values, but, the combination of these columns must be unique across the database table.

Syntax

Following is the syntax to create an SQL Composite Key while creating a table −

```
CREATE TABLE table_name(
   column1 datatype,
   column2 datatype,
   column3 datatype,
```

```
   .....
   columnN datatype,
   CONSTRAINT composite_key_name,
   PRIMARY KEY(column_name)
);
```

Here, the **composite_key_name** is the optional placeholder which holds the name of a Composite Key in a table. It is used while dropping the constraint from a table in some databases.

Example

In the following example, we are creating a table named CUSTOMERS with multiple columns. The Composite Key is created when a PRIMARY KEY is defined on ID and NAME columns together. Look at the query below –

```
CREATE TABLE CUSTOMERS(
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2),
   CONSTRAINT ck_customers
   PRIMARY KEY (ID, NAME)
);
```

Where, ck_customers is the name of a composite key of this table.

Output

Following is the output of the above statement −

Query OK, 0 rows affected (0.02 sec)

Verification

As we have created a Composite Key on the columns ID and NAME of the CUSTOMERS table, the combination of values in these columns can not be duplicated. To verify it, let us insert two records with same values in these columns into the CUSTOMERS table −

Open Compiler

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),

(1, 'Ramesh', 25, 'Delhi', 1500.00 );

You can observe that the second INSERT statement generates an error message saying **"Duplicate entry"** as shown below −

ERROR 1062 (23000): Duplicate entry '1-Ramesh' for key 'customers.PRIMARY'-

## Dropping a Composite Key in MySQL

You can drop the composite key from a table in MySQL database using the **ALTER TABLE... DROP** statement.

Syntax

Following is the syntax to drop the Composite Key in MySQL −

ALTER TABLE table_name DROP PRIMARY KEY;

Example

Using the following SQL statement, we can drop the Composite Key constraint from the CUSTOMERS table −

ALTER TABLE CUSTOMERS DROP PRIMARY KEY;

Output

The above SQL statement produces the following output −

Query OK, 1 row affected (0.02 sec)

Records: 1  Duplicates: 0  Warnings: 0

Verification

Since, we have dropped the composite from the CUSTOMERS table, so now you can insert the duplicate values in the columns ID and NAME.

Let us insert two records with the same ID and NAME into the CUSTOMERS table −

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 25, 'Delhi', 1500.00 ),

(1, 'Ramesh', 23, 'Kota', 2000.00 );

If you retrieve the contents the CUSTOMERS table you can find the records with same ID and NAME as −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 1 | Ramesh | 25 | Delhi | 1500.00 |
| 1 | Ramesh | 23 | Kota | 2000.00 |

## Dropping a Composite Key in SQL Server

In SQL Server, we have a different syntax to drop a composite key of a table. The syntax is almost similar, but we just need to specify the composite key name in order to drop it, rather than the keyword PRIMARY KEY.

Syntax

Following is the syntax to drop a composite key in SQL Server −

ALTER TABLE table_name DROP composite_key_name;

Example

Assuming that a composite key "ck_customers" is created on ID and NAME columns of the CUSTOMERS table, we will use the following query to drop it −

ALTER TABLE CUSTOMERS DROP ck_customers;

Output

When we execute the above query, the composite key will be dropped.

Commands completed successfully.

Verification

To verify whether we have removed the composite key from the CUSTOMERS table or not, insert duplicate values into the ID and NAME columns using the following query −

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 25, 'Delhi', 1500.00 ),

(1, 'Ramesh', 23, 'Kota', 2000.00 );

As we can see in the table below, both the customers have the same ID and NAME −

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmedabad | 2000.00 |
| 1 | Ramesh | 25 | Delhi | 1500.00 |
| 1 | Ramesh | 23 | Kota | 2000.00 |

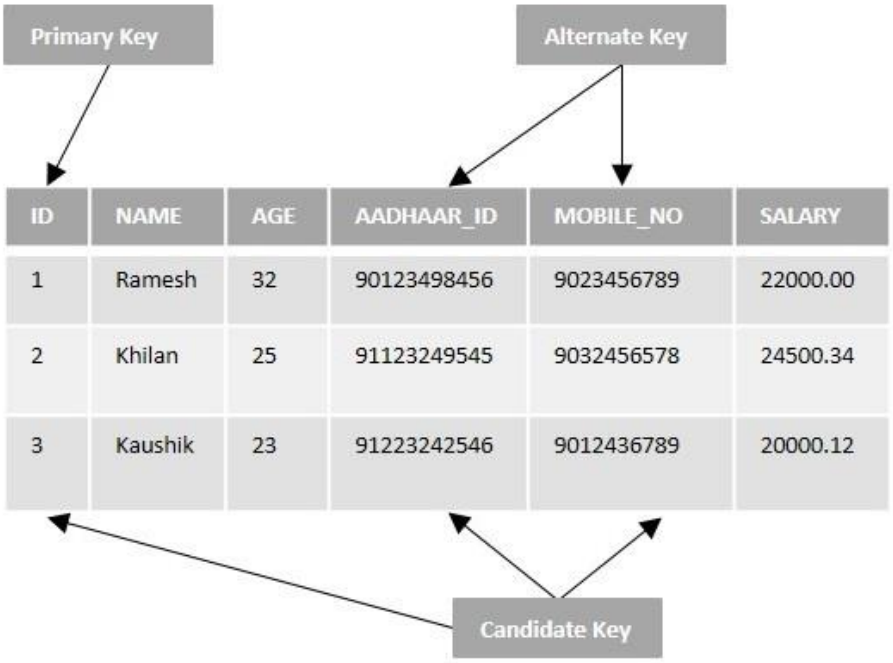
## The SQL Alternate Key

SQL **Alternate Keys** in a database table are candidate keys that are not currently selected as a primary key. They can be used to uniquely identify a tuple(or a record) in a table.

There is no specific query or syntax to set the alternate key in a table. It is just a column that is a close second candidate which could be selected as a primary key. Hence, they are also called secondary candidate keys.

*If a database table consists of only one candidate key, that is treated as the primary key of the table, then there is no alternate key in that table.*

Let us understand the concept of alternate key with an example. Suppose we have a table named CUSTOMERS with various fields like ID, NAME, AGE, AADHAAR_ID, MOBILE_NO and SALARY as shown below.




| ID | NAME | AGE | AADHAAR_ID | MOBILE_NO | SALARY |
|---|---|---|---|---|---|
| 1 | Ramesh | 32 | 90123498456 | 9023456789 | 22000.00 |
| 2 | Khilan | 25 | 91123249545 | 9032456578 | 24500.34 |
| 3 | Kaushik | 23 | 91223242546 | 9012436789 | 20000.12 |

The details like id, mobile number and aadhaar number of a customer are unique, and we can identify the records from the CUSTOMERS table uniquely using their respective fields; ID, AADHAAR_ID and MOBILE_NO. Therefore, these three fields can be treated as candidate keys.

And among them, if one is declared as the primary key of the CUSTOMERS table then the remaining two would be alternate keys.

Features of Alternate Keys

Following are some important properties/features of alternate keys −

- The alternate key does not allow duplicate values.
- A table can have more than one alternate keys.
- The alternate key can contain NULL values unless the NOT NULL constraint is set explicitly.
- All alternate keys can be candidate keys, but all candidate keys can not be alternate keys.

- The primary key, which is also a candidate key, can not be considered as an alternate key.

Example

For a better understanding, let us create the above discussed table demonstrating the usage of the various keys and illustrating the fields that can be considered as alternate keys.

Open Compiler

```
CREATE TABLE CUSTOMERS(
   ID INT,
   NAME VARCHAR (20),
   AGE INT,
   AADHAAR_ID BIGINT,
   MOBILE_NO BIGINT,
   SALARY DECIMAL (18, 2),
   PRIMARY KEY(ID)
);
```

Now, insert some records into the CUSTOMERS table using the INSERT statement as shown below −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 90123498456, 9023456789, 22000.00 ),
(2, 'Khilan', 25, 91123249545, 9032456578, 24500.34 ),
(3, 'Kaushik', 23, 91223242546, 9012436789, 20000.12 );
```

The table will be created as −

| ID | NAME | AGE | AADHAAR_ID | MOBILE_NO | SALARY |
|---|---|---|---|---|---|
| 1 | Ramesh | 32 | 90123498456 | 9023456789 | 22000.00 |
| 2 | Khilan | 25 | 91123249545 | 9032456578 | 24500.34 |

| | | | | | |
|---|---|---|---|---|---|
| 3 | Kaushik | 23 | 91223242546 | 9012436789 | 20000.12 |

Keys in a table

As a summary lets revisit all the keys in a database table −

## Candidate Key

A **Candidate key** is a subset of super keys that is used to uniquely identify records of a table. It can be a single field or multiple fields. The primary keys, alternate keys, foreign keys in a table are all types of candidate key.

## Primary Key

A **Primary Key** is a main key that is used to retrieve records from a table. It is a single column or field in a table that uniquely identifies each record in a database table.

It can be set using the PRIMARY KEY keyword while creating a table using the CREATE TABLE statement. Following is the basic syntax to create primary key constraint on a column in a table −

```
CREATE TABLE table_name(
   column1 datatype,
   column2 datatype,
   column3 datatype,
   .....
   columnN datatype,
   PRIMARY KEY(column_name)
);
```

## Foreign Key

The Primary key of one table will be the **Foreign key** in another table. While inserting values into these tables, values in the primary key field must match the values in the foreign key field; otherwise, the foreign key column will not accept the **INSERT** query and throws an error.

In SQL server, the syntax to set a foreign key field in a table is –

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ...
    CONSTRAINT fk_name
        FOREIGN KEY (column_name)
        REFERENCES referenced_table(referenced_column)
);
```

## Alternate Key

An **Alternate key** is a candidate key that could be a primary key but is not. Like primary key, it also uniquely identifies the records in a field of a table to retrieve row tuples from the said table. There can be a single or multiple fields identifying as alternate keys in a table.